

Gate concepts: separation, coast, anchor, warm-up

Working notes from the design discussion, in question-and-answer form. `docs/HOST_DECISIONS.md` is the contract; this file explains the quantities it uses. Line numbers refer to the current `SPF_protectedNav.m`.

1. Separation and "the coast of the reference"

- **Coast of a reference epoch:** the filter's estimate at that epoch, propagated forward by the INS only (Φ), with no GNSS update. It is what the solution would be today if no measurement had been applied since the reference. Paper E28.
- **Host solution:** the same start plus every GNSS correction accepted since. Each correction is an increment $dx = K*y$ applied at its epoch, and from then on it rides along the INS dynamics like everything else in the state.
- **Separation** = host solution now minus coast of the reference = the sum of the increments since the reference, each propagated from its own epoch to now.

Epoch	Increment $K*y$	Host solution	Coast of reference	Separation
0		2.0	2.0	0
1	+0.5	2.5	2.0	0.5
2	+0.3	2.8	2.0	0.8
3	-0.1	2.7	2.0	0.7

(one axis, $\Phi = 1$ for position; with velocity coupling an increment's contribution keeps changing after it is applied, which is why each one is Φ -propagated)

2. Why the gate keeps the difference, not two states (E28 in increment form)

The paper keeps the filter state and the coast state and subtracts them at the end. The gate keeps only the difference:

- filter: $x_{KF}(k) = \Phi * x_{KF}(k-1) + dx(k)$
- coast (E28): $x_C(k) = \Phi * x_C(k-1)$
- subtract: $d(k) = \Phi * d(k-1) + dx(k)$

This is E28 subtracted from the filter's own recursion: every term of E28 is there, applied to the difference, plus the one term the coast lacks. In a closed-loop error-state filter there is no single absolute state to subtract from (the estimate is split between `stateFB` and `states`, and the split depends on the feedback gain). The increment $K*y$ is the same for any split, so the difference form is the only one that is invariant to how the host stores its estimate (checked by `tests/test_feedback_split.m`). The coast covariance line next to it is E31 unchanged.

3. P_C and P_{KF}

- **P_{KF} :** the host filter's covariance now, P_+ after this epoch's update (`kfMeas.postCov`). Shrinks or holds when measurements arrive.
- **P_C :** the coast covariance, the reference epoch's P_+ propagated with Φ and Q only, never updated. Only grows. Each open window carries its own P_C from the epoch it opened; the anchor carries its own too.
- **$P_C - P_{KF}$** is what GNSS has contributed since the reference and, for a consistent linear filter, the covariance of the separation. The SS test uses its diagonal on each monitored axis: $\sigma_{SS} = \sqrt{P_C - P_{KF}}$, alarm when $|d| > k_{FA} * \sigma_{SS}$.

	Variance	Sigma
P_C after 5 s of coast (Q adds 0.2 m ² /s)	5 m ²	2.24 m
P_{KF} after five GNSS updates	1 m ²	1 m
$P_C - P_{KF}$ (separation)	4 m ²	2 m
SS threshold $k_{FA} * \sigma_{SS}$		10.5 m

Consequences: the threshold is not a fixed distance (tolerant under poor geometry, tight under strong GNSS; the test asks whether the pull is plausible, not whether it is big). A zero or negative $P_C - P_{KF}$ on an axis makes the test undefined on

that axis: no alarm, no division. In COAST the operational filter is not updated, so P_C and P_{KF} coincide and the coast covariance is the filter's own prior; that is what re-validation compares the raw innovation against.

4. Windows and the anchor

- One window opens every alarm-free armed epoch (line 147) and lives `WINDOW_LENGTH = 10` epochs, so about ten are open at once and one closes every epoch. A window's separation starts at zero at its open epoch and is dropped at its close.
- The anchor uses the same recursion (lines 99 to 101, every armed `NOMINAL` epoch), so it is always "live": subtracting it from today's x_+ gives the anchor's coast at today, nothing to replay.
- **Refresh**: a window that closes with no alarm over its whole life replaces the anchor with its own separation and covariance and stamps it with the close epoch (lines 114 to 119). In clean operation the anchor is replaced every epoch; it is the most recent certified window, not a particular one. Its reference is the opening epoch of that window, so a latch also removes that window's increments: one window length (5 s) more conservative than strictly necessary, deliberately (a below-threshold spoof starting inside the window could have touched its last increments). Anchoring at the close instead is a one-line change (zero the separation at the refresh, keep the window's P_+).
- **Warm-up** (unarmed): re-seeded every epoch with separation zero, covariance P_+ , stamp now (lines 87 to 91).
- **Commit** after probation: separation zero, covariance the trial's P_+ , stamp the commit epoch (lines 245 to 249).
- **COAST and PROBATION**: not propagated; at the latch its covariance became the coast covariance and the operational filter's own prior carries it.
- **Age limit**: checked only at an alarm (line 124). If the anchor is older than `MAX_ANCHOR_AGE` or not valid (lines 135 to 138): no correction, `info.anchorMissing`, COAST from the current (possibly contaminated) solution with the current P_+ as coast covariance. No new anchor until the next commit. In practice hard to reach now: the age grows only while no window closes clean, and in `NOMINAL` that means alarms, which already latch.

5. Warm-up arming

Code: `SPF_protectedNav.m` lines 66 to 85 (counter), 87 to 91 (unarmed action), constants `CST_spfParam.m` lines 71 to 73, telemetry `info.armed`.

- After a gate (re)initialisation the host filter is not converged: initial P , zero error state (the receiver clock included), few rows, interval matrices just reset. Its first corrections are far larger than its own covariance predicts, and the SS test reads that as a pull.
- Clearing the persistent data does not avoid this; it causes it. "Cleared" and "converged" are different states.
- In a full run the transient happens during alignment, when the gate is inactive (`navActive = false`), so the first active epoch sees a settled filter. A segment started mid-flight with a manual reset, or an emergency reset, starts filter and gate together on a cold filter.
- The monitors therefore arm only after `ARM_EPOCHS` qualifying epochs, with `numMeas >= REVAL_MIN_MEAS` and `K_MISSED_DETECTION * sigma_pos(P+) < ARM_PL_MAX` (100 m). A PL above the limit restarts the count (the filter is diverging or coasting); a row dropout with the PL still inside only pauses it. Arming is sticky until the next re-initialisation. The simulation references use 10 epochs; the host uses 120 (60 s), see section 6.
- Ten epochs is a count, not a proof of convergence; the PL condition is the real guard. In the mode figure the warm-up is the grey band, which must cover the whole transient; if the SS margins are still near 1 when the band ends, raise `ARM_EPOCHS` or lower `ARM_PL_MAX`.
- Any reset that sends the KF back to its initial P (emergency reset, re-alignment) must reset the gate too, through `firstCall` or by dropping `navActive` during the reset. Otherwise the anchor, windows and trial carry pre-reset history against a restarted filter and a latch is almost guaranteed.

6. What the jammed data set showed (470 s GNSS outage, host reset at re-acquisition)

- The host's protective reset fires when GNSS returns and the position differs from the INS by more than its limit; it re-seeds the filter with the initial P . Everything the gate alarmed on in the first runs was the convergence after that reset (clock, velocity and biases estimated from scratch): SS margins of 5, CPI 20, NIS 33 on the first post-reset epoch.
- The covariance during the outage itself was honest: σ_N from P_+ was 11.7 km against a true drift of 22 km (ratio 1.9), σ_E 11.7 km against 2.4 km. Q is not the problem on this data; the earlier reading that P had not grown enough was wrong, it was the post-reset P .

- Two things fixed it: the gate is reset together with the host (`navActive` dropped in the reset and alignment modes), and the warm-up is long enough for the post-reset bias convergence, `ARM_EPOCHS = 120` (60 s) in the host configuration instead of the simulation default of
- With both, the non-shadow run matches the shadow run: one host reset, no latch, errors at zero after re-acquisition.
- Side effect of the first arming rule: an epoch with fewer than `REVAL_MIN_MEAS` rows restarted the count, so on data with early dropouts the first warm-up lasted 150 s instead of 60 s. The rule now pauses the count on a row dropout and restarts it only when the PL leaves the limit.
- After the reset the filter is still settling 110 s later: a short GNSS gap at 970 s gave SS margins of 0.85. Not an alarm, but the margin to watch on runs that start from a reset.

7. Protective reset versus the gate

- The host's protective reset (GNSS-vs-INS position difference beyond a limit: reset and re-align from GNSS) and the gate react to the same event. Once the reset also re-initialises the gate, a step spoof beyond the limit is accepted by the reset before the gate can act, the gate warms up on the spoofed GNSS, and the end of the spoof is a second reset. The spoof is never seen.
- The two are told apart by the covariance, which is what the gate tests. A jump after a long outage sits inside an honest P (22 km against a sigma of 11.7 km in the jammed data set: no alarm) and the reset is the right answer. A step spoof on a converged filter is tens of sigma: alarm, latch, GNSS held off.
- Wiring rule: the reset check runs after the gate on the same epoch and is skipped while `info.resetInhibit` is true (gate in COAST or PROBATION, or alarming this epoch). `COAST_BUDGET_EPOCHS` bounds the hold: beyond it `info.coastBudgetExceeded` is raised and the host may reset, with integrity not assured across that reset. See `docs/HOST_2HZ_WIRING.m` step 6.
- Limitation: during the warm-up the gate cannot alarm, so a step spoof in that first minute after a start or reset still goes through the reset.

8. GNSS present but not applied (validity flag false)

- Test: 2000 s of clean data with the host's GNSS validity flag forced false from 500 to 1500 s. The host went to pure INS, but the 2 Hz function still ran the scratch update and handed the gate 20 to 30 rows with the scratch `x+` and `P+` every epoch.
- What the gate saw: a filter that "accepted" 25 rows each epoch while the operational solution stayed on the drifting INS. The increments it accumulates are the corrections the host never applied, so they grow with the INS drift, and after about 110 s the separation exceeded the SS threshold: a latch on clean data. Then latch / probation / commit cycles, because the host ignored the hand-back as well; in shadow mode a probation / veto oscillation every few epochs, because no trial exists.
- Rule (already in `docs/HOST_DECISIONS.md`, "invalid update" row): the gate sees what the filter used. Not applied means `numMeas = 0`, `xPost = xPrior`, `PPost = PPrior`. With that, an outage of this kind is the jammed data set again: NOMINAL throughout, PL growing, and at re-acquisition either a jump inside an honest P (no alarm) or the host's reset with the gate warming up after it.
- Shadow mode only exercises NOMINAL; its COAST and PROBATION verdicts are not meaningful.
- Repeated with the host's actual behaviour (the 2 Hz function, and so the gate, is not called at all while the receiver delivers nothing): NOMINAL throughout, identical with and without the gate, a warm-up after the reset at re-acquisition, no latch. The gate sees the outage as one long interval through `propTel`; all its counters are in calls, not seconds.